

Enigma Machine in Python

Gajanth Sabaratnam and Alexander Wollesen

December 19, 2025

1 Solutions

The solutions for the second part of the final project (implementing function "encrypt" and function "find_rotors" are in the FinalEnigmaModel.py file, underneath the "rotor_clicked" function.

2 Purpose of the functions

The purpose of **encrypt** is to present an Enigma style of encryption process. It requires two variables:

Rotors: A 3-length string for example "BPD". with each representing one of the rotors.

Message: A string based on the alphabet A-Z, written in all capital letters.

The function returns the encrypted string produced by initially adjusting the rotors correctly for each letter and also sending the letter through the fast/medium/slow rotor wiring, reflector and then back out

The purpose of the **find rotors** function is to discover the settings the rotors were adjusted to when encrypting a given message. It requires two variables:

Message: Same as previously mentioned

Cipher: The result of running a message through the **encrypt** function, a string based on the alphabet A-Z, comprised fully of capital letters.

The function brute forces it's way through every possible combination of the fast, medium, and slow rotor. Each combination of rotor settings is used to encrypt the message that was provided, and if the resulting cipher is the same as the cipher provided, the correct rotor settings have been found and they are returned as a string.

3 Challenges

The main challenge with the **encrypt** function was finding a way to execute the rotor rotations, without needing to type each letter of the message individually, unlike what was done in part 1 with the enigma keyboard. After brainstorming we decided the best way was to implement a **for loop**, which was executed for every letter in the message. Thus we were able to reuse a lot of the old code from part 1, mainly the section which was used to encrypt a letter and rotate the rotors, greatly simplifying our coding process. We had to ensure that the rotor rotations happened at the correct time and order. The encryption depends on the rotor positions at the moment a letter is processed so therefore even a small mistake in the rotor rotation would change the whole cipher. We had to ensure that the medium rotor only advanced after the fast rotor had completed a entire rotation, and the same for the slow rotor in relation to the medium. We resolved these issues by using numeric representation of letters and modulo arithmetic:

$pos = (ord(current) - ord('A') + 1) \bmod 26$

$self.rotors[2] = chr(pos + ord('A'))$

Here $ord(current) - ('A')$ converts the rotor position from a character to a numeric value

and advances the rotor by one while `mod26` ensures that when the rotor moves past 'Z' it wraps back to 'A'.

Next was the problem of connecting the letters together to form a single string. To overcome this, we used the `.append` and `.join` methods. At the start of the function, an empty list called "result" is created to store the letters as they get encrypted. The `.append` method adds the encrypted letter to the list after they have been encrypted, adding them from left to right. Once all letters are encrypted and added to the list, the `.join` method joins together every element of the list into a string. the " " before the `.join` method ensures that there are no gaps between the letters.

The hardest part of coding the `find_rotors` function was imagining how to find the rotors with just the message and encrypted message. However, while running some tests to ensure that the `encrypt` function worked as intended, we thought of simply reusing said function to reverse-engineer our code and find the rotors used. The next step was finding a way to test all rotor combinations, but with the use of `for loops` this was made trivial.

4 Testing and Debugging

For the rotors, we printed rotor states after each character so we could verify correct rotor "rollover" behavior. We also verified encryption through testing whether the Enigma machine had the expected output through these commands:

```
model = EnigmaModel()
rotor_original = "A-Z + A-Z + A-Z"
message = "ANY STRING OF CAPITAL LETTERS"
cipher = model.encrypt(rotor_original, message)
print(cipher), we would then ensure that the encrypted message printed is the same as the
sequence of lamps that light up when writing the message into the interactive enigma key-
board
rotor = model.find_rotors(message, cipher)
rotor_original == rotor, if value is True we know it works correctly.
```